

Практическая работа

Длительность: 90 минут (1 пара)

Формат: индивидуальная работа, пошаговое выполнение

Результат: работающее приложение-задачник с middleware, DI, контроллером, маршрутизацией и model binding

Шаг 0. Подготовка

Проверить, что установлено:

Если не установлено — скачать .NET SDK 8 с dotnet.microsoft.com.

Создать проект:

```
dotnet new mvc -n TaskBoard
cd TaskBoard
dotnet run
```

Открыть <http://localhost:5000> — должна загрузиться стартовая страница ASP.NET Core.

Шаг 1. Первый middleware: заголовок и логирование

Открыть Program.cs. Найти строку `var app = builder.Build();`. После неё, до `app.UseStaticFiles()`, добавить:

```
app.Use(async (context, next) =>
{
    context.Response.Headers.Append("X-App-Name", "TaskBoard");

    var logger = context.RequestServices.GetRequiredService<ILogger<Program>>();
    logger.LogInformation("--> {Method} {Path}", context.Request.Method,
context.Request.Path);

    await next(context);

    logger.LogInformation("<-- {StatusCode}", context.Response.StatusCode);
});
```

Проверить:

1. Запустить `dotnet run`
2. Открыть <http://localhost:5000>
3. F12 → Network → клик на запрос → Headers → найти X-App-Name: TaskBoard
4. В консоли (где запущен `dotnet run`) — увидеть логи `--> GET /` и `<-- 200`

Усложнение: добавить middleware, который при запросе к `/health` сразу возвращает 200 OK и не передаёт дальше:

```
app.Use(async (context, next) =>
{
    if (context.Request.Path == "/health")
    {
        context.Response.StatusCode = 200;
        await context.Response.WriteAsync("healthy");
        return; // не вызываем next — запрос не идёт дальше
    }
    await next(context);
});
```

Проверить: /health возвращает «healthy», остальные запросы работают как раньше.

Шаг 2. Замер времени обработки

Добавить **ещё один** middleware до предыдущего (ближе к началу конвейера):

```
app.Use(async (context, next) =>
{
    var sw = Stopwatch.StartNew();
    await next(context);
    sw.Stop();

    context.Response.Headers.Append("X-Response-Time-ms",
sw.ElapsedMilliseconds.ToString());
});
```

Не забыть using System.Diagnostics; вверху файла.

Проверить:

1. Перезапустить
2. В заголовках ответа — X-Response-Time-ms: 12 (или другое число)

Вопрос себе: почему этот middleware стоит до логирующего? Что будет, если поставить **после**?
Проверить.

Шаг 3. Создание сервиса и DI

3.1. Создать интерфейс и реализацию:

Создать папку Services и файл Services/ITaskService.cs:

```
namespace TaskBoard.Services;

public class TaskItem
{
    public int Id { get; set; }
    public string Title { get; set; } = "";
    public string? Description { get; set; }
    public bool IsDone { get; set; }
    public DateTime CreatedAt { get; set; } = DateTime.Now;
}

public interface ITaskService
{
    List<TaskItem> GetAll();
    TaskItem? GetById(int id);
    TaskItem Add(string title, string? description);
    bool MarkDone(int id);
    bool Delete(int id);
}
```

Создать Services/InMemoryTaskService.cs:

```
namespace TaskBoard.Services;

public class InMemoryTaskService : ITaskService
{
    private readonly List<TaskItem> _tasks = new();
    private int _nextId = 1;

    public List<TaskItem> GetAll() => _tasks.ToList();
```

```

public TaskItem? GetById(int id) => _tasks.FirstOrDefault(t => t.Id == id);

public TaskItem Add(string title, string? description)
{
    var task = new TaskItem
    {
        Id = _nextId++,
        Title = title,
        Description = description,
        IsDone = false
    };
    _tasks.Add(task);
    return task;
}

public bool MarkDone(int id)
{
    var task = GetById(id);
    if (task == null) return false;
    task.IsDone = true;
    return true;
}

public bool Delete(int id)
{
    var task = GetById(id);
    if (task == null) return false;
    _tasks.Remove(task);
    return true;
}
}

```

3.2. Зарегистрировать в DI:

В Program.cs, до builder.Build(), добавить:

```
builder.Services.AddSingleton<ITaskService, InMemoryTaskService>();
```

Вопрос себе: почему Singleton, а не Scoped? Что будет при перезапуске приложения? Проверить.

Шаг 4. Первый контроллер

4.1. Создать контроллер:

Создать Controllers/TasksController.cs:

```

using Microsoft.AspNetCore.Mvc;
using TaskBoard.Services;

namespace TaskBoard.Controllers;

public class TasksController : Controller
{
    private readonly ITaskService _service;

    public TasksController(ITaskService service)
    {
        _service = service;
    }

    // GET /tasks
    public IActionResult Index()

```

```

{
    var tasks = _service.GetAll();
    return View(tasks);
}

// GET /tasks/details/1
public IActionResult Details(int id)
{
    var task = _service.GetById(id);
    if (task == null) return NotFound();
    return View(task);
}

// GET /tasks/create
public IActionResult Create()
{
    return View();
}

// POST /tasks/create
[HttpPost]
public IActionResult Create(string title, string? description)
{
    if (string.IsNullOrEmpty(title))
    {
        ModelState.AddModelError("title", "Заголовок обязателен");
        return View();
    }

    _service.Add(title, description);
    return RedirectToAction(nameof(Index));
}

// POST /tasks/done/1
[HttpPost]
public IActionResult Done(int id)
{
    _service.MarkDone(id);
    return RedirectToAction(nameof(Index));
}

// POST /tasks/delete/1
[HttpPost]
public IActionResult Delete(int id)
{
    _service.Delete(id);
    return RedirectToAction(nameof(Index));
}
}

```

4.2. Проверить маршрутизацию:

В Program.cs убедиться, что есть:

```

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");

```

Шаг 5. Представления (Razor)

5.1. Создать папки и файлы:

```
Views/  
└─ Tasks/  
    └─ Index.cshtml  
    └─ Details.cshtml  
    └─ Create.cshtml
```

5.2. Views/Tasks/Index.cshtml:

```
@model List<TaskBoard.Services.TaskItem>  
@{  
    ViewData["Title"] = "Список задач";  
}  
  
<h1>Задачи</h1>  
  
<a href="/tasks/create" class="btn btn-primary">Новая задача</a>  
  
@if (!Model.Any())  
{  
    <p>Пока нет задач. Создайте первую!</p>  
}  
else  
{  
    <table class="table mt-3">  
        <thead>  
            <tr>  
                <th>ID</th>  
                <th>Заголовок</th>  
                <th>Статус</th>  
                <th>Дата</th>  
                <th>Действия</th>  
            </tr>  
        </thead>  
        <tbody>  
            @foreach (var task in Model)  
            {  
                <tr>  
                    <td>@task.Id</td>  
                    <td>  
                        <a href="/tasks/details/@task.Id">@task.Title</a>  
                    </td>  
                    <td>  
                        @if (task.IsDone)  
                        {  
                            <span class="badge bg-success">Готово</span>  
                        }  
                        else  
                        {  
                            <span class="badge bg-warning">В работе</span>  
                        }  
                    </td>  
                    <td>@task.CreatedAt.ToString("dd.MM HH:mm")</td>  
                    <td>  
                        @if (!task.IsDone)  
                        {  
                            <form method="post" action="/tasks/done/@task.Id"  
                                class="d-inline">  
                                <button type="submit" class="btn btn-sm btn-outline-  
success">✓</button>  
                            </form>  
                        }  
                    </td>  
                </tr>  
            }  
        </tbody>  
    </table>  
}
```

```

        <form method="post" action="/tasks/delete/@task.Id" class="d-
inline"
        onsubmit="return confirm('Удалить?')">
        <button type="submit" class="btn btn-sm btn-outline-
danger">X</button>
    </form>
</td>
</tr>
}
</tbody>
</table>
}

```

5.3. Views/Tasks/Details.cshtml:

```

@model TaskBoard.Services.TaskItem
@{
    ViewData["Title"] = "Детали задачи";
}

<h1>@Model.Title</h1>

<p><strong>Статус:</strong> @(Model.IsDone ? "Готово" : "В работе")</p>
<p><strong>Описание:</strong> @(Model.Description ?? "-")</p>
<p><strong>Создана:</strong> @Model.CreatedAt</p>

<a href="/tasks" class="btn btn-secondary">Назад</a>

```

5.4. Views/Tasks/Create.cshtml:

```

@{
    ViewData["Title"] = "Новая задача";
}

<h1>Новая задача</h1>

<form method="post" action="/tasks/create">
    <div class="mb-3">
        <label class="form-label">Заголовок</label>
        <input type="text" name="title" class="form-control" />
        @if (ViewData.ModelState["title"].Errors.Count > 0)
        {
            <div class="text-danger">
                @ViewData.ModelState["title"].Errors[0].ErrorMessage
            </div>
        }
    </div>
    <div class="mb-3">
        <label class="form-label">Описание</label>
        <textarea name="description" class="form-control" rows="3"></textarea>
    </div>
    <button type="submit" class="btn btn-primary">Создать</button>
    <a href="/tasks" class="btn btn-secondary">Отмена</a>
</form>

```

5.5. Добавить ссылку в меню:

Открыть Views/Shared/_Layout.cshtml. Найти навигацию и добавить:

```

<a class="nav-link" href="/tasks">Задачи</a>

```

Проверить:

1. Открыть /tasks — пустой список
 2. Нажать «Новая задача» → создать → увидеть в списке
 3. Нажать «✓» → статус меняется на «Готово»
 4. Нажать «X» → задача удаляется
 5. Кликнуть на заголовок → страница деталей
-

Шаг 6. Конечная точка API — JSON

Добавить в TasksController методы, которые отдают JSON вместо HTML:

```
// GET /tasks/api/list
[HttpGet("tasks/api/list")]
public IActionResult ApiList()
{
    var tasks = _service.GetAll();
    return Json(tasks);
}

// GET /tasks/api/1
[HttpGet("tasks/api/{id:int}")]
public IActionResult ApiGet(int id)
{
    var task = _service.GetById(id);
    if (task == null) return NotFound();
    return Json(task);
}

// POST /tasks/api/create
[HttpPost("tasks/api/create")]
public IActionResult ApiCreate([FromBody] CreateTaskRequest request)
{
    if (string.IsNullOrEmpty(request.Title))
        return BadRequest(new { error = "Заголовок обязателен" });

    var task = _service.Add(request.Title, request.Description);
    return CreatedAtAction(nameof(ApiGet), new { id = task.Id }, task);
}
```

Добавить класс для запроса (можно прямо в ITaskService.cs или отдельно):

```
public class CreateTaskRequest
{
    public string Title { get; set; } = "";
    public string? Description { get; set; }
}
```

Проверить через терминал:

```
# Получить список
curl http://localhost:5000/tasks/api/list

# Создать задачу
curl -X POST http://localhost:5000/tasks/api/create \
  -H "Content-Type: application/json" \
  -d '{"title":"Тест через curl","description":"Создано из терминала"}'

# Получить по ID
curl http://localhost:5000/tasks/api/1
```

Шаг 7. Проверка через DevTools

1. Открыть /tasks в браузере
2. F12 → вкладка **Network**
3. Обновить страницу
4. Кликнуть на запрос /tasks
5. Проверить:
 1. **Request Headers** — метод GET, путь /tasks
 2. **Response Headers** — X-App-Name: TaskBoard, X-Response-Time-ms: ...
 3. **Response** — HTML-код страницы
6. Создать задачу через форму → найти POST-запрос в Network
7. Кликнуть на него → проверить:
 1. **Method**: POST
 2. **Form Data**: title=...&description=...
 3. **Status**: 302 (redirect)
8. Вызвать curl `http://localhost:5000/tasks/api/list` → проверить JSON-ответ

Чек-лист для сдачи

Для оценки необходимо показать:

#	Что показать	Где
1	Проект запускается	dotnet run, <code>http://localhost:5000</code>
2	Кастомные заголовки в ответе	DevTools → Network → Headers
3	Логи запросов в консоли	Терминал, где запущен dotnet run
4	Создание задачи через форму	<code>/tasks/create</code>
5	Список задач	<code>/tasks</code>
6	Отметка выполненной	кнопка ✓
7	Удаление задачи	кнопка ✕
8	JSON API — список	<code>curl /tasks/api/list</code>
9	JSON API — создание	<code>curl -X POST /tasks/api/create</code>
10	Ответ на вопрос «почему сервис Singleton»	устно

Выполнить задания (все, начиная с легкого - на оценку 5):

Уровень 1 — Лёгкое:

Добавить на страницу /tasks счётчик: «Всего задач: X, выполнено: Y». Обновлять в контроллере, передавать во View через ViewBag.

Уровень 2 — Среднее:

Добавить middleware, который при запросах к `/tasks/api/*` проверяет заголовок X-API-Key. Если заголовка нет или он не равен `secret123` — возвращать 401 Unauthorized. API-методы должны работать только с правильным ключом.

Уровень 3 — Сложное:

Добавить фильтрацию: query-параметр `?status=all|done|active` на странице /tasks. В контроллере фильтровать список. Сохранять выбранный фильтр в ссылке, чтобы он не сбрасывался при отметке/удалении.
